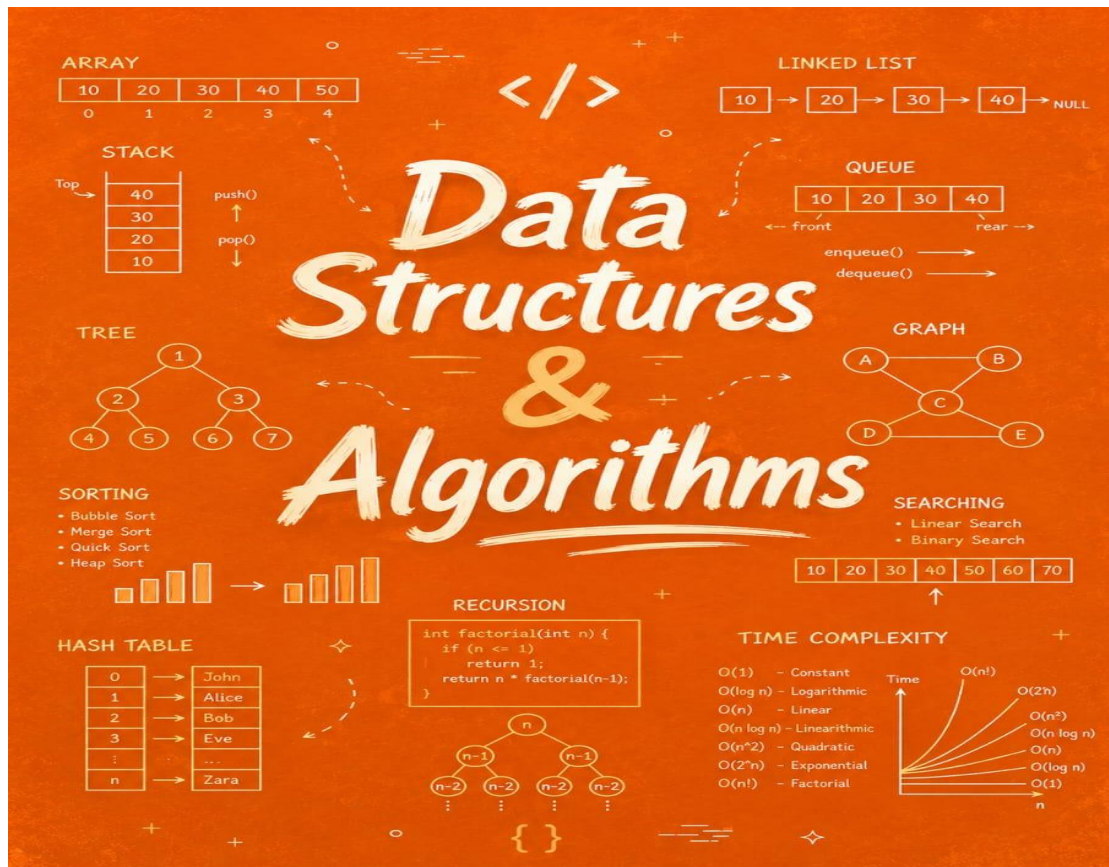




Java Data Structures and Algorithms

By Jawad Jammoul

Content:

Chapter 01 : LinkedList	Page
	1
Chapter 02 : Stacks and Queues	7
Chapter 03 : Algorithms Analysis	11
Chapter 04 : Recursion	14
Chapter 05 : Tree	15
Chapter 06 : Graphs	28
Chapter 07 : HashTable & HashMap	38
Chapter 08 : Vector	40

Chapter 01: LinkedList

01*Definition:

A LinkedList is a linear data structure where each element (called a node) points to the next one. It is a dynamic structure, meaning it can grow or shrink in size during runtime.

02*Why using LinkedList:

Dynamic size: Unlike arrays, it doesn't need a fixed size.

Efficient insertions / deletions: Adding or removing nodes is faster (especially in the middle or beginning).

No memory waste. Uses memory only as needed (no pre-allocated size).

03*Each Node Consists of Two Parts:

Data part: Stores the actual value (e.g., int, String, etc.)

Link (Reference) part: Points to the next node in the list.

04*NodeData Class:

```
class NodeData {  
    private int value;  
    public NodeData(int value) {  
        this.value = value;  
    }  
    public int getValue() {  
        return value;  
    }  
    public void setValue(int value) {  
        this.value = value;  
    }  
    public int compareTo(NodeData x) {  
        if(this.value > x.value)  
            return 1;  
        else if(this.value < x.value)  
            return -1;  
        else  
            return 0;  
    }  
}
```

05*Node Class:

```
public class Node {
    private NodeData data;
    private Node next;
    public Node(NodeData data) {
        this.data = data;
        this.next = null;
    }
    public NodeData getData() {
        return data;
    }
    public void setData(NodeData data) {
        this.data = data;
    }
    public Node getNext() {
        return next;
    }
    public void setNext(Node next) {
        this.next = next;
    }
}
```

06*LinkedList Class:

```
public class LinkedList {
    private Node head;
    //Constructor
    public LinkedList() {
        this.head = null;
    }
    //Getter
    public Node getHead() {
        return head;
    }
    //Setter
    public void setHead(Node head) {
        this.head = head;
    }
}
```

print() : Used to display all elements in the list in order

```
public void print() {
    Node current = head;
    while(current!=null) {
        System.out.print(current.getData().getValue() + "->");
        current = current.getNext();
    }
    System.out.println("null");
}
```

isEmpty() : Checks if the list is empty

```
public boolean isEmpty() {  
    return head == null;  
}
```

countNodes() : Returns the total number of nodes in the list

```
public int countNodes() {  
    int count = 0;  
    Node current = head;  
    while(current!=null) {  
        count++;  
        current = current.getNext();  
    }  
    return count;  
}
```

belongs(NodeData data) : Checks if a specific value exists in the list

```
public boolean belongs(NodeData data) {  
    Node current = head;  
    while(current != null) {  
        if(current.getData().compareTo(data) == 0) {  
            return true;  
        }  
        current = current.getNext();  
    }  
    return false;  
}
```

addHead(NodeData data) : Adds a new node at the beginning of the list

```
public void addHead(NodeData data) {  
    Node newNode = new Node(data);  
    newNode.setNext(head);  
    head = newNode;  
}
```

addTail(NodeData data) : Adds a new node at the end of the list

```
public void addTail(NodeData data) {  
    Node newNode = new Node(data);  
    if(head == null) {  
        head = newNode;  
        return;  
    }  
    Node current = head;  
    while(current.getNext() != null) {  
        current = current.getNext();  
    }  
    current.setNext(newNode);  
}
```

insertPosition(NodeData data , int pos): Inserts a new node at a specific position in the list

```
public void insertPosition(NodeData data , int pos) {
    if(pos<=0) {
        addHead(data);
        return;
    }
    Node current = head;
    int index = 0;
    while(current!=null && index<pos-1) {
        current = current.getNext();
        index++;
    }
    Node newNode = new Node(data);
    newNode.setNext(current.getNext());
    current.setNext(newNode);
}
```

equals(LinkedList other) : Compares two lists to check if they are equal in structure and data

```
public boolean equals(LinkedList other) {
    Node current1 = this.head;
    Node current2 = other.head;
    while(current1 != null && current2 != null) {
        if(current1.getData().compareTo(current2.getData()) != 0) {
            return false;
        }
        current1 = current1.getNext();
        current2 = current2.getNext();
    }
    return current1 == null && current2 == null ;
}
```

addImplace(LinkedList other): Appends another list to the end of the current list

```
public void addImplace(LinkedList other) {
    if(this.head == null)
        this.head = other.head;
    else {
        Node current = this.head;
        while(current.getNext() != null) {
            current = current.getNext();
        }
        current.setNext(other.head);
    }
}
```

copy() : Creates and returns a new list that is a copy of the current one

```
public LinkedList copy() {
    LinkedList newList = new LinkedList();
    if(head == null)
        return newList;
    Node current = head;
    while(current!=null) {
        newList.addTail(new NodeData(current.getData().getValue()));
        current = current.getNext();
    }
    return newList;
}
```

reverse() : Reverses the order of the nodes in the list

```
public void reverse() {
    Node prev = null ;
    Node current = head;
    while(current!=null) {
        Node next = current.getNext() ;
        current.setNext(prev);
        prev = current ;
        current = next;
    }
    head = prev;
}
```

isPal() : Checks if the list is a palindrome (same forward and backward)

```
public boolean isPal() {
    LinkedList reversed = this.copy();
    reversed.reverse() ;
    return this.equals(reversed);
}
```

merges(LinkedList other) : Merges two sorted lists into a new sorted list

```
public LinkedList merges(LinkedList other) {
    LinkedList result = new LinkedList();
    Node p1 = this.head;
    Node p2 = other.head;
    while(p1 != null && p2 != null) {
        if(p1.getData().compareTo(p2.getData())<=0) {
            result.addTail(new NodeData(p1.getData().getValue()));
            p1 = p1.getNext() ;
        }
        else {
            result.addTail(new NodeData(p2.getData().getValue()));
            p2 = p2.getNext() ;
        }
    }
    while(p1 != null) {
        result.addTail(new NodeData(p1.getData().getValue()));
        p1 = p1.getNext();
    }
    while(p2 != null) {
        result.addTail(new NodeData(p2.getData().getValue()));
        p2 = p2.getNext();
    }
    return result;
}
```

deletePosition(int pos) : Deletes the node at a specific position

```
public void deletePosition(int pos) {
    if(pos<0 || head == null)
        return;
    if(pos == 0) {
        deleteHead();
        return;
    }
    Node current = head;
    for(int i =0 ; current.getNext() != null && i< pos-1 ; i++) {
        current = current.getNext() ;
    }
    if(current.getNext() != null) {
        current.setNext(current.getNext().getNext()) ;
    }
}
```

deleteHead() : Removes the first node of the list

```
public void deleteHead() {
    if(head != null) {
        head = head.getNext();
    }
}
```

deleteTail() : Removes the last node of the list

```
public void deleteTail() {
    if(head == null || head.getNext() == null) {
        head = null;
        return;
    }
    Node current = head;
    while(current.getNext().getNext() != null) {
        current = current.getNext();
    }
    current.setNext(null);
}
```

07*Application class:

```
public class Application {
    public static void main(String[] args) {
        //Create first LinkedList
        LinkedList list1 = new LinkedList();
        list1.addHead(new NodeData(10));
        list1.addTail(new NodeData(20));
        list1.addTail(new NodeData(30));
        list1.insertPosition(new NodeData(25), 2);
        list1.print();
        list1.deleteTail();
        list1.print();
        list1.deletePosition(1);
        list1.print();
        list1.reverse();
        list1.print();
        LinkedList copyList = list1.copy();
        System.out.println("Copy equals original ?" + list1.equals(copyList)); //true
        list1.addTail(new NodeData(10));
        list1.addTail(new NodeData(20));
        list1.print();
        System.out.println("Is Palindrome?" + list1.isPal());
        LinkedList list2 = new LinkedList();
        list2.addTail(new NodeData(5));
        list2.addTail(new NodeData(15));
        copyList.addImplace(list2);
        copyList.print();
        LinkedList sorted1 = new LinkedList();
        sorted1.addTail(new NodeData(1));
        sorted2.addTail(new NodeData(3));
        LinkedList sorted2 = new LinkedList();
        sorted2.addTail(new NodeData(2));
        sorted2.addTail(new NodeData(4));
        LinkedList merged = sorted1.merges(sorted2);
        System.out.print("Merged List: ");
        merged.print();
    }
}
```


Output: 10 -> 20 -> 25 -> 30 -> null isEmpty: false Count : 4 Belongs 25? true 20 -> 25 -> null; 20 -> null 20 -> null Copy equals original ? true 20 -> 5 -> 15 -> null Merged List : 1 -> 2 -> 3 -> 4 -> null

Chapter 02 : Stack and Queues

01*ADT:

An Abstract Data Type (ADT) is a specification of a collection of data and the operations (methods) that can be performed on it

The definition of an ADT only mentions what operations are to performed but not how these operations will implemented. The process of providing only the essentials and hiding the details is known as abstraction

A data structure is the implementation for an ADT

02*Stack Array Implementation:

push(int value) : add an item to the top of the stack

pop() : remove an item from the top of the stack

top() or peek() : returns the top item without removing it

isEmpty() : returns true if the stack is empty

isFull() : checks whether the stack has reached its maximum size

```

class Stack {
    private int[] arr;
    private int top;
    public Stack(int size) {
        arr = new int[size];
        top = -1;
    }
    public void push(int value) {
        if(isFull()) {
            System.out.println("Stack is empty");
            return -1;
        }
        else
            return arr[top--];
    }
}
public int top() {
    if(isEmpty()) {
        System.out.println("Stack is empty");
        return -1;
    }
    else
        return arr[top];
}
public boolean isEmpty() {
    return top == -1 ;
}
public boolean isFull() {
    return top == arr.length - 1;
}
}

```

03*Stack LinkedList:

```
public class Stack {
    private Node top;
    public Stack() {
        top = null;
    }
    public boolean isEmpty() {
        return top == null;
    }
    public void push(NodeData item) {
        Node newNode = new Node(item);
        newNode.next = top;
        top = newNode;
    }
    public NodeData pop() {
        if(isEmpty()) {
            System.out.println("Stack is empty");
            return null;
        }
        else {
            NodeData value = top.data;
            top = top.next;
            return value;
        }
    }
    public NodeData top() {
        if(isEmpty()) {
            System.out.println("Stack is empty");
            return null;
        }
        else {
            return top.data;
        }
    }
}

class Node {
    NodeData data;
    Node next;
    public Node(NodeData data) {
        this.data = data;
        this.next = null;
    }
}
```

Advantage of use Stack LinkedList:

- No fixed size limit , it can grow dynamically as needed.
- Faster insertion and deletion at the top (O(1) time).
- No need to manage array resizing or shifting elements.
- Efficient memory usage since no unused space is allocated.
- Easier to implement recursion-like behavior with linked nodes.
- Can store complex data types using custom NodeData objects.

04*Implementing a queue using an array:

A queue is a waiting line or a linear list in which items are added at one end and deleted from the other end.

Familiar examples are queues at a bank , a supermarket , etc. People are supposed to join the queue at the tail and exit from the head

Queue , as the name suggests is used whenever we need to manage any group of objects in an order in which the first one coming in , also gets out first , also gets out first (First In First Out or simply FIFO) while the others wait for their turn

These are the basic operations we want to perform on a queue:

- Add an item to the queue (we say enqueue)
- Take an item off the queue (we say dequeue)
- Check whether the queue is empty
- Inspect the item at the head of the queue

enqueue(int item) : Adds an element to the end of the queue if it's not full

dequeue() : removes and returns the front element of the queue

isEmpty() : Checks if there are no elements in the queue

isFull() : Checks if the queue has reached its maximum capacity

```
class Queue {
    private int[] queueArray ;
    private int head , tail ;
    public Queue() {
        this(10);
    }
    public Queue(int size) {
        queueArray = new int[size];
        head = 0;
        tail = -1;
    }
    public void enqueue(int item) {
        if(isFull()) {
            System.out.println("Queue is full");
        }
        else {
            queueArray[++tail] = items ;
        }
    }
    public int dequeue() {
        if(isEmpty()) {
            System.out.println("Queue is empty");
            return -1;
        }
        else {
            return queueArray[head++];
        }
    }
    public boolean isEmpty() {
        return head>tail ;
    }
    public boolean isFull() {
        return tail == queueArray.length - 1;
    }
    public int head() {
        if(isEmpty()) {
            System.out.println("Queue is empty");
            return -1;
        }
        else {
            return queueArray[head];
        }
    }
}
```

```

}
public int tail() {
    if(isEmpty()) {
        System.out.println("Queue is empty");
        return -1;
    }
    else {
        return queueArray[tail];
    }
}
}
}

```

05*Queue LinkedList:

Code for the Queue ADT using Nodes

We will use the same classes NodeData and Node used in the implementation of the LinkedList class and the Stack class

The class Queue has two data fields head and tail , both are references to Node

isEmpty() : Checks if the queue has no elements

enqueue(NodeData item) : Adds a new elements to the end of the queue

dequeue() : Removes and returns the front element of the queue

head() : returns the data of the first node in the queue

tail() : returns the data of the last node in the queue

```

class Queue {
    private Node head,tail;
    public Queue() {
        head = tail = null ;
    }
    public boolean isEmpty() {
        return head == null ;
    }
    public void enqueue(NodeData item) {
        Node newNode = new Node(item) ;
        if(isEmpty()) {
            head = tail = newNode ;
        }
        else {
            tail.next = newNode;
            tail = newNode;
        }
    }
    public NodeData dequeue() {
        if(!isEmpty()) {
            System.out.println("Queue is empty");
            return null;
        } else {
            NodeData value = head.data;
            head = head.next;
            if(head == null) {
                tail = null;
            }
            return value;
        }
    }
    public NodeData head() {
        if(isEmpty()) {
            System.out.println("Queue is empty");
            return null;
        } else {

```

```

return head.data;
}
}
public NodeData tail() {
    if(isEmpty()) {
        System.out.println("Queue is empty");
        return null;
    } else {
        return tail.data;
    }
}
}
class Nodes {
    NodeData data;
    Node next;
    public Node(NodeData data) {
        this.data = data;
        this.next = null;
    }
}
}

```

Chapter 03 : Algorithms Analysis

01*What is an Algorithm:

An algorithm is a finite , step-by-step procedure to solve a specific problem . It takes input , processes it , and produces an output. Algorithms must be

Finite: Ends after a number of steps

Definite: Each step is clearly defined

Effective: Each step is doable

Example: A simple algorithm to find the maximum number in an array

02*Algorithm Analysis:

Algorithm analysis is the process of evaluating the efficiency of an algorithm, mainly in terms of:

Time complexity(how fast it runs)

Space complexity (how much memory it uses)

03*Running Time:

Running time means the actual time an algorithm takes to execute based on:

Input size

Type of input

Computer speed

However, instead of measuring in seconds , we analyze it using primitive operations (assignments , comparisons , loops , etc)

04*Experimental Study:

This involves running the program with different inputs and measuring execution time using code (like in Java)

But it's limited because:

Results vary on different machines

Doesn't predict performance for very large inputs

05*Measuring Running Time Experimentally in Java:

You can measure execution time in Java using:

```
long start = System.nanoTime();  
//your code here  
long end = System.nanoTime();  
System.out.println("Running Time" + (end-start) + "ns");
```

This gives time in nanoseconds

06*Challenges of Experimental Analysis:

Hardware dependent.

Input dependent : Different inputs may result in different running times

Limited scalability : Can't easily test with huge inputs

07*Moving Beyond Experimental Analysis:

We use theoretical models:

Count primitive operations (basic steps like + , - , = , < , etc.)

Relate those to input size n

Use functions to express how the number of steps grows with n

08*Counting Primitive Operations:

Primitive operations include:

Assignments

Comparisons

Arithmetic operations

Function calls

Array indexing

Example:

```
int sum =0;  
for(int i=0;i<n ;i++)  
    sum+=i;
```

This runs n times , each doing 1 addition and 1 assignment -> Total : 2n primitive ops

09*Best , Worth , and Average Case:

Best Case : Minimum steps

Worst Case: Maximum steps

Average Case : Expected number of steps across all inputs

Example - Linear Search

Best Case: item in first => $O(1)$

Worst Case: item in last or missing => $O(n)$

Average Case => $O(n)$

10*The Seven Functions Used in the Analysis of Algorithms:

These functions describe growth rates - how fast the number of steps increases with n

1.Constant Function - $O(1)$:

Always takes the same time , regardless of input.

Example:

```
int x = 5;
```

Time doesn't grow with n

2. Logarithmic Function - $O(\log n)$:

Input size gets divided each time

Example:

```
while(n>1)
    n = n/2;
```

Used in Binary Search

3. Linear Function - $O(n)$:

Steps grow directly with input size

Example:

```
for(int i =0;i<n;i++)
    sum+=arr[i];
```

4. Linear Arithmetic Function - $O(n \log n)$:

Appears in efficient sorting algorithms like Merge Sort and Quick Sort (on average).

5. Quadratic Function - $O(n^2)$:

Used in algorithms with nested loops over the input

Example:

```
for(int i=0;i<n;i++)
    for(int j=0;j<n;j++)
        sum +=(i+j);
```

Common in simple sorting like Bubble Sort

6. Cubic Function - $O(n^3)$:

Three nested loops

Example:

```
for(int i=0;i<n;i++)
    for(int j=0;j<n;j++)
        for(int k=0; k<n ;k++)
            doSomething();
```

7. Exponential Function - $O(2^n)$:

Time doubles with each increase in n .

Example:

Recursive Fibonacci

```
int fib(int n) {
    if(n<=1)
        return n;
    return fib(n-1) + fib(n-2) ;
}
```

Extremely slow for large n

11* Comparing Growth Rates:

From fastest to slowest:

$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n)$

Lower growth = better scalability

12* Comparative Analysis:

Helps choose the best algorithm by comparing their time complexities

Example:

Linear Search: $O(n)$

Binary Search : $O(\log n)$

Use Binary Search for sorted data because it's faster

13*Examples of Algorithm Analysis:

1.Constants & Loops:

```
for(int l=0 ; l<n ;l++)  
    for(int j=0; j<5 ; j++)  
        //do something
```

This is $O(n)$ because the inner loop is constant (5)

2.Finding the Maximum in an Array:

```
int max = arr[0];  
for(int l = 1 ; l< arr.length ;l++) {  
    if(arr[l]>max)  
        max = arr[l];  
}
```

Time complexity : $O(n)$ - goes through each element once

Chapter 04 : Recursion

01*What is Recursion?

Recursion is a programming technique where a method calls itself to solve a problem by breaking it into smaller , simpler instances of the same problem

02*Components of a Recursive method:

Every recursive method must have:

Base Case: A condition to stop the recursion . Without it , the method would call itself infinitely

Recursive Call: A call the same method with a smaller or simpler input that moves the problem closer to the base case

03*Example - Factorial using Recursion (returns int):

```
public static int factorial(int n) {  
    if(n==0 || n==1)  
        return 1;  
    return n*factorial(n-1);  
}
```

Base Case: if $n==0$ or $n==1$, return 1.

Recursive Case: Multiply n by the result of $\text{factorial}(n-1)$

04*Void vs Non-Void Recursive Methods:

A.Non-Void Recursion (returns a value):

These methods return a result. For example:

Here, the method returns the final result after all recursive calls

```
public static int sum(int n) {  
    if(n==1)  
        return 1;  
    return n + sum(n-1);  
}
```


B.Void Recursion(does not return a value):

These methods perform actions like printing or modifying data , without returning anything

```
public static void countdown(int n) {  
    if(n==0)  
        return;  
    System.out.println(n);  
    countdown(n-1);  
}
```

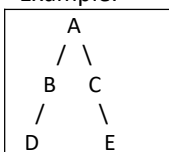
Chapter 05: Tree

01*Trees Definitions:

1.Tree:

A non-linear data structure made of nodes

Example:



2.Root:

The topmost node of the tree

Example:

in the above tree , node A is the root

3.Leaf:

A node that has no children

Example:

Nodes C , D and E are leaves

4.Branch:

A path from the root to any leaf

Example:

One branch is A -> B -> D

5.Weight:

The total number of nodes in a tree or subtree

Example:

Full tree: 5 Nodes(A,B,C,D,E) -> weight= 5

Subtree starting at B : nodes B, D, E -> weight = 3

6.Tree:

Distance from the root node

Example:

A is at level 0

B and C are at level 1

D and E are at level 2

7.Height:

The longest path from the root to a leaf (in number of edges)

Example:

A -> B -> D = 2 edges -> height = 2

8.Parent:

A node that has child nodes

Example:

A is parent of B and C
B is parent of D and E

9.Child:

A node that comes from a parent node

Example:

B is child of A
E is child of B

10.Sibling:

Nodes that share the same parent

Example:

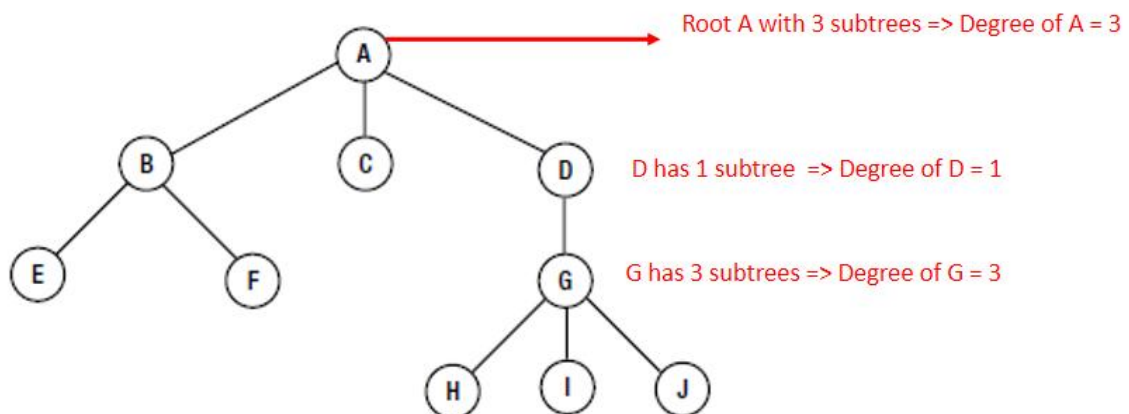
D and E are siblings (they have same parent B)
B and C are siblings (parent is A)

11.Degree:

The degree of a node is the number of subtrees of the node. Think of it as the number of lines leaving the node.

For example

degree(A) = 3 , degree(B) = 2 , degree(C) = 0 , degree(D) = 1 , and degree(G) = 3



02*Binary Trees:

1.Binary Trees:

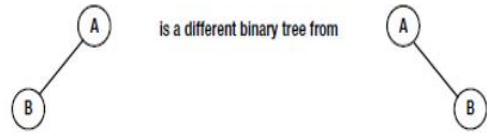
In a binary tree , a node may have 0 , 1 or 2 children maximum

Example 1:

Here's a binary tree with one node, the root:

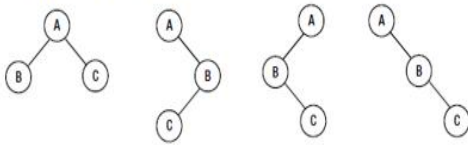


Example 2:



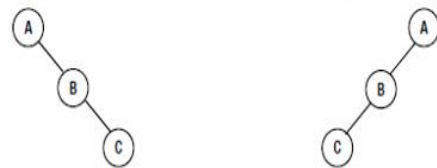
Example 3:

Here are binary trees with three nodes:

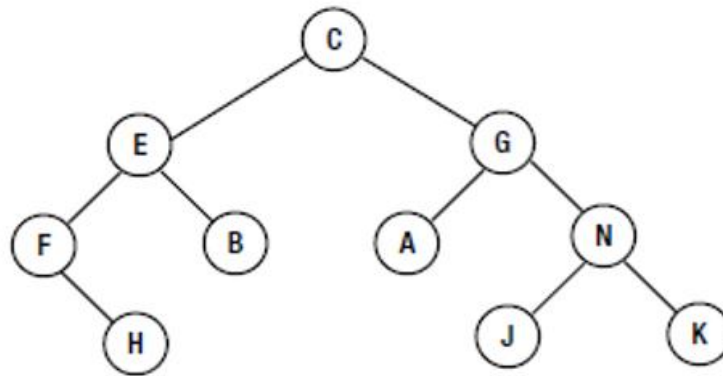


Example 4:

Here are binary trees with all left subtrees empty and all right subtrees empty:

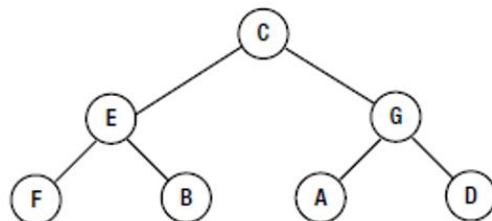


Example 5: General Binary Tree



Example 5: A complete binary tree

A complete binary tree is a binary tree where each node, except the leaves, has exactly two subtrees.

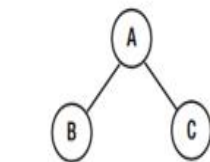


2.Traversing a Binary Tree:

Pre-order:

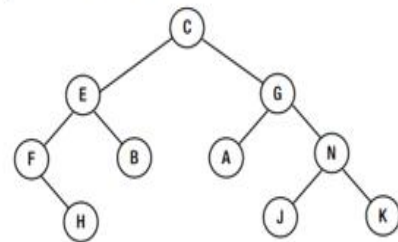
- 1.Visit the root
- 2.Traverse the left subtree in pre-order
- 3.Traverse the right subtree in pre-order

The *pre-order* traversal of this tree



is A B C.

The *pre-order* traversal of this tree



is C E F H B G A N J K.

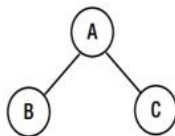
preOrder Traversal -> Node Left Right (NLR):

```
public void preOrder() {  
    preOrderTraversal(root);  
}  
private void preOrder(TreeNode current) {  
    if(current!=null) {  
        System.out.print(current.data + ' ');  
        preOrderTraversal(current.left);  
        preOrderTraversal(current.right);  
    }  
}
```

Inorder:

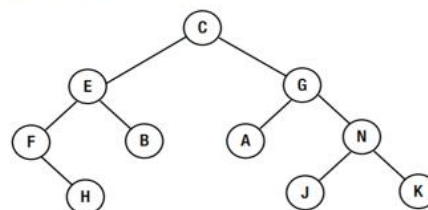
- 1.Traverse the left subtree in in-order
- 2.Visit the root
- 3.Traverse the right subtree in in-order

The *in-order* traversal of this tree



is B A C.

The *in-order* traversal of this tree



is F H E B C A G J N K

```

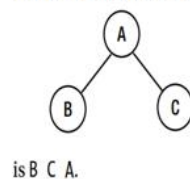
inOrder Traversal -> Left Node Right (LNR)
public void inOrder() {
    inOrderTraversal(root);
}
private void inOrderTraversal(TreeNode current) {
    if(current!=null) {
        inOrderTraversal(current.left);
        System.out.print(current.data + " ");
        inOrderTraversal(current.right);
    }
} //end inOrderTraversal

```

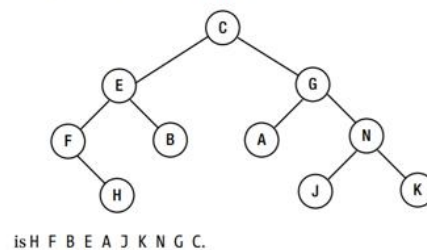
Post-order:

- 1.Traverse the left subtree in post-order
- 2.Traverse the right subtree in post-order
- 3.Visit the root

The *post-order* traversal of this tree



The *post-order* traversal of this tree



postOrder Traversal -> Left Right Node (LRN)

```

public void postOrder() {
    postOrderTraversal(root);
}
private void postOrderTraversal(TreeNode current) {
    if(current !=null) {
        postOrderTraversal(current.left);
        postOrderTraversal(current.right);
        System.out.print(current.data + " ");
    }
}

```

How to represent a Binary Tree:

Each node of a binary tree consists of three fields:

- .A field containing the data at the node
- .A reference to the left subtree
- .A reference to the right subtree

We can begin by writing the class `TreeNode` which is abit similar to the class `Node` used with the `LinkedList` class

```

class TreeNode {
    NodeData data;
    TreeNode left, right;
    public TreeNode(NodeData d) {
        data = d;
        left = right = null;
    }
}
class NodeData {
    int value;
    public NodeData(int v) {
        value = v;
    }
}

```

```

}
class BinaryTree {
    private TreeNode root;
    public BinaryTree() {
        root = null;
    }
    //other methods
}

```

03*Binary Search Trees (BST):

1.A Binary Search Tree (BST) is a binary tree which has the following properties:

The left sub-tree of a node has a key less than to its parent node's key

The right sub-tree of a node has a key greater than to its parent node's key

In other words:

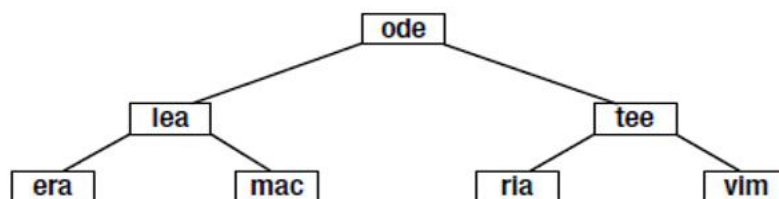
For every node x in the tree , if y is in that node's left subtree , then $y < x$

For every node x in the tree, if y is in that node's right subtree , then $y > x$

2.BST Example:

Given any node , a word in the left subtree is "smaller" and a word in the right subtree is "greater" than the word at the node . (Here , smaller and greater refer to alphabetical order based on the first letter.)

It facilitates the search for a given key using a method of searching similar to the binary search of an array



Code for the search operation:

```

public boolean search(NodeData item) {
    TreeNode current = root;
    if(isEmpty())
        System.out.print("The search operation failed. The BST is empty");
    else
        while(current!=null) {
            if(item.compareTo(current.data) == 0)
                return true;
            else if(item.compareTo(current.data) < 0)
                current = current.left;
            else
                current = current.right;
        }
    return false;
}

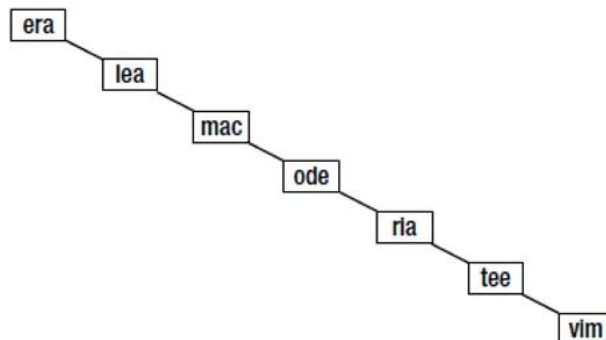
```

The worst case(Degenerate Tree):

If the words come in alphabetical order , then the tree build will be like this

Searching such a tree is reduced to a sequential search of a linked list

This kind of tree is called a degenerate tree



Inserting an element into a BST:

In the previous slides we covered the method search used to search for an item in a binary search tree (BST)

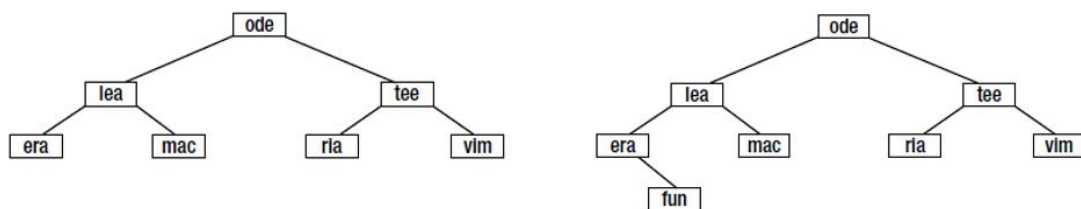
In the next slides we look at the method findOrInsert used to insert an item a binary search tree (BST) if it is not already found there

Adding the Non Existing word "fun":

When explaining the search algorithm , we discovered that if fun is to exist in the BST it must be on the right of era

Since it is not there, if we need to add it, then it must be added to the right of era

Not only does the binary search tree facilitates searching, but if an item is not found , it can easily inserted



```
public void findOrInsert(NodeData item) {
    TreeNode current = root, parent = null, newNode;
    boolean found = false;
    if(isEmpty())
        root = new TreeNode(item);
    else {
        while(current!=null) {
            if(item.compareTo(current.data) == 0) {
                System.out.println("Insert Operation failed.The element is already in the BST");
                found =true;
                break;
            }
            else if(item.compareTo(current.data) < 0) {
                parent = current;
                current = current.left;
            }
            else {
                parent = current;
                current = current.right;
            }
        }
        if(found == false) {
            newNode = new TreeNode(item);
            if(parent != null) {
                if(item.compareTo(parent.data) < 0)
                    parent.left = newNode;
                else
                    parent.right = newNode;
            }
            else
                root = newNode;
        }
    }
}
```

```

current = current.right;
}
}
if(!found) {
    newNode = new TreeNode(item);
    if(item.compareTo(parent.data) < 0)
        parent.left = newNode;
    else
        parent.right = newNode;
}
}
}

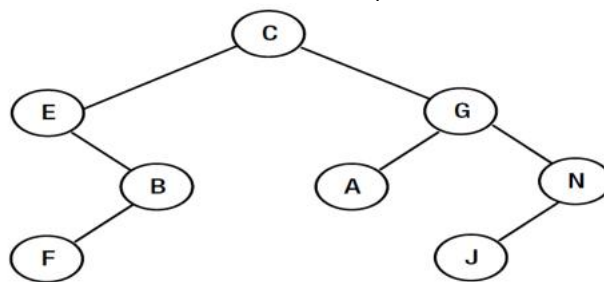
```

04*Level-Order Traversal:

An alternative to pre-order, in-order, and post-order traversals is level-order

We traverse the tree level by level (from left to right in each level)

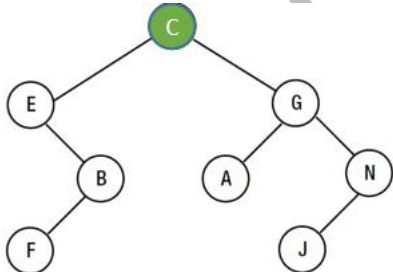
To perform a level-order traversal, we use a queue of `TreeNode` objects



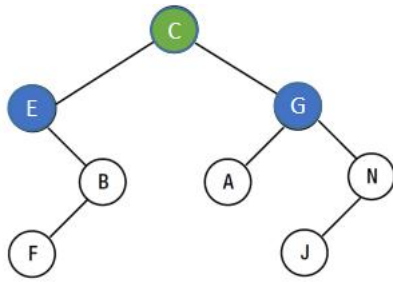
Its level-order traversal is C E G B A N F J.

Level-Order Traversal using a queue

Enqueue C in Q

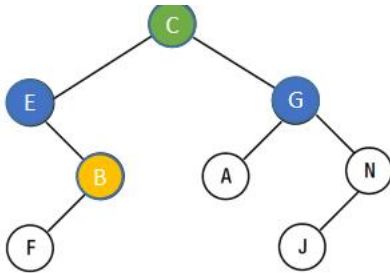


Q is not empty, so dequeue and visit (output) C, then add (enqueue) its children E and G to Q



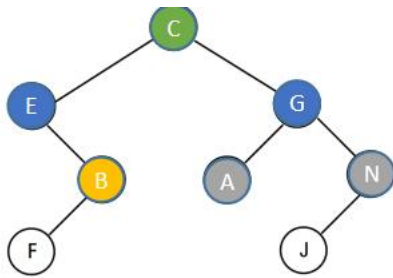
Output: C

Q is not empty , so dequeue and visit (output) E, then add its child B to Q



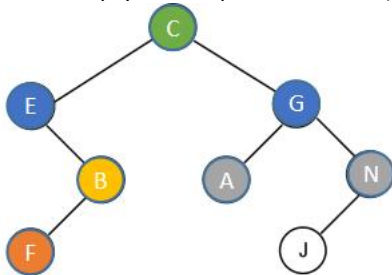
Output: C E

Q is not empty , so dequeue and visit (output) G , then add its children A and N to Q



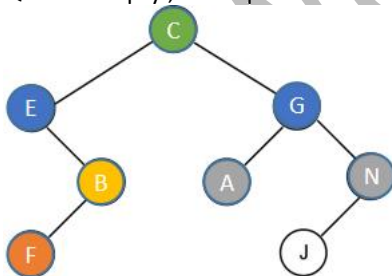
Output: C E G

Q is not empty , so dequeue and visit (output) B , then add its child F to Q



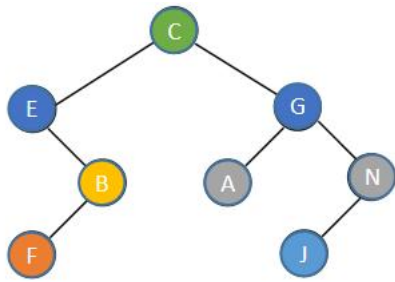
Output: C E G B

Q is not empty , so dequeue and visit (output) A . Since A has no children them nothing is added to Q



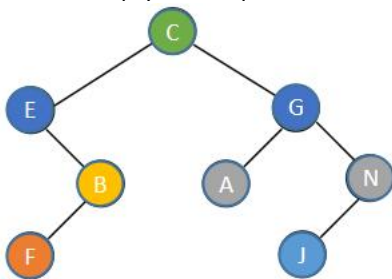
Output: C E G B A

Q is not empty , so dequeue and visit (output) N , then add its child J to Q



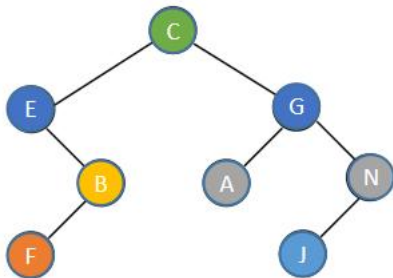
Output: C E G B A N

Q is not empty , so dequeue and visit (output) F . Since F has no children then nothing is added to Q



Output: C E G B A N F

Q is not empty , so dequeue and visit (output) J . Since J has no children then nothing is added to Q



Output: C E G B A N F J

Now Q is empty . The traversal ends having visited the nodes in the order C E G B A N F J

05*Some useful tree functions:

Count the number of nodes in a tree:

```
public int numNodes() {  
    return countNodes(root);  
}  
private int countNodes(TreeNode current) {  
    if(current == null)  
        return 0;  
    return 1 + countNodes(current.left) + countNodes(current.right);  
}
```

Note: the public method is not the recursive one

Count the number of leaves in the tree . That is the number of nodes that have their left and right nodes both null.

Some Useful Binary Tree Functions:

```
public int numLeaves() {  
    return countLeaves(root);  
}  
private int countLeaves(TreeNode current) {  
    if(current == null)  
        return 0;  
    if(current.left == null && current.right == null)  
        return 1;  
    return countLeaves(current.left) + countLeaves(current.right);  
}
```

Note: Same recursion principle as the countNode method

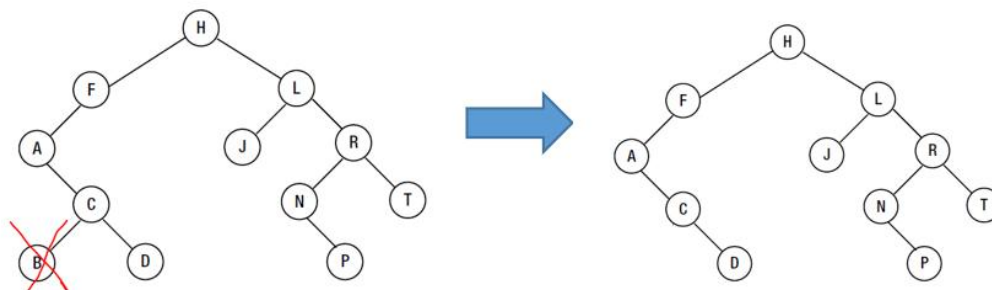
Calculate the height (number of levels) of the tree:

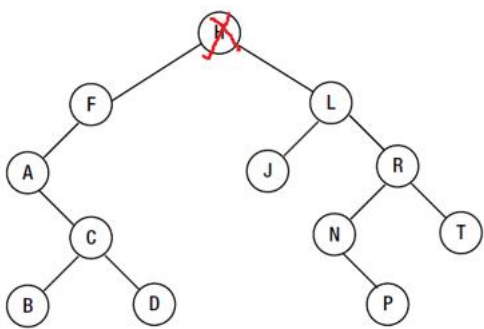
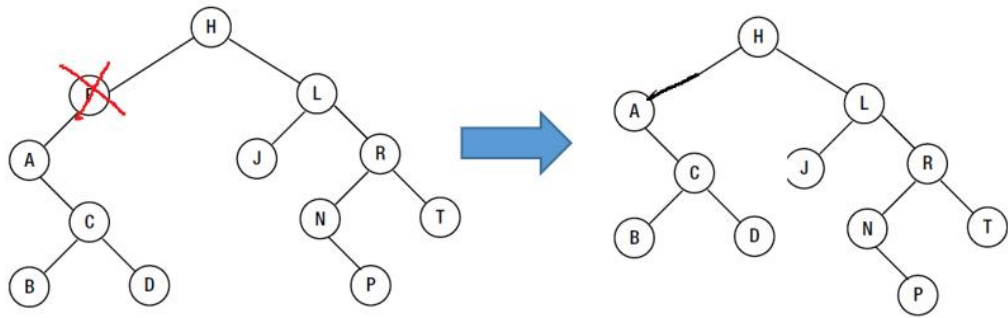
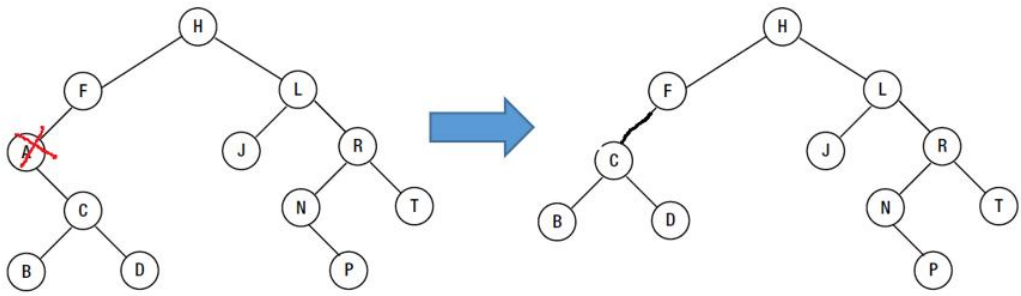
```
public int height() {  
    return countLevels(root);  
}  
private int countLevels(TreeNode current) {  
    if(current == null)  
        return 0;  
    return 1 + Math.max(countLevels(current.left), countLevels(current.right));  
}
```

06*BST deletion:

1.replace H with J called in-order successor then delete the successor

2.replace H with F called in-order predecessor then delete the predecessor





Code for method delete:

```
public void delete(NodeData item) {
    TreeNode parent = null , child , temp , current = root ;
    boolean found = false;
    if(root == null) {
        System.out.println("Delete operation failed. The BST is empty.");
        return ;
    }
    while(current != null && !found) {
        if(item.compareTo(current.data) == 0)
            found = true;
        else if(item.compareTo(current.data) < 0) {
            parent = current;
            current = current.left;
        }
        else {
            parent = current ;
            current = current.right;
        }
    }
    if(!found)
        System.out.println("Delete failed. Element is not in the BST");
    else {
        if(current.left != null && current.right != null) {
            temp = current;
            current = current.left;
        }
        temp.data = current.data;
    }
    child = current.left;
    if(child == null)
        child = current.right;
    if(current == root)
        root = child;
    else if(parent.left == current)
        parent.left = child;
    else
        parent.right = child;
}
```

Chapter 06: Graphs

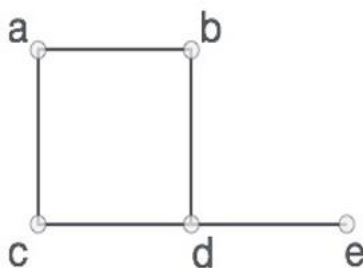
01*Graph Terminology:

A graph is a representation of a set of objects where some pairs of objects are connected by links . The interconnected objects are represented by points termed as vertices (nodes) , and the links that connect the vertices are called edges

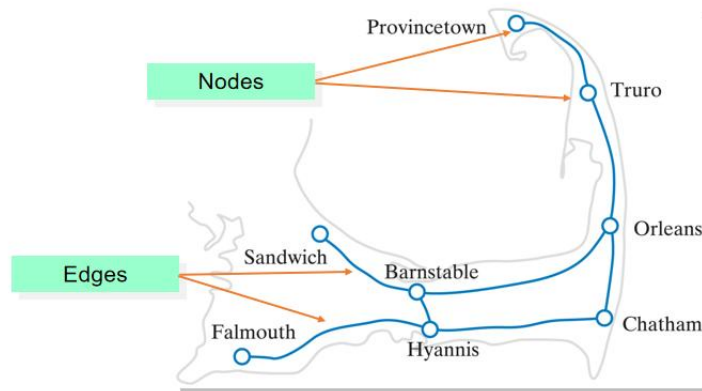
Formally, a graph is a pair of sets (V,E), where V is the set of vertices and E is the set of edges, connecting the pairs of vertices. Take a look at the following graph

$V = \{a, b, c, d, e\}$

$E = \{ab, ac, bd, cd, de\}$



Graph Example(Road Map):



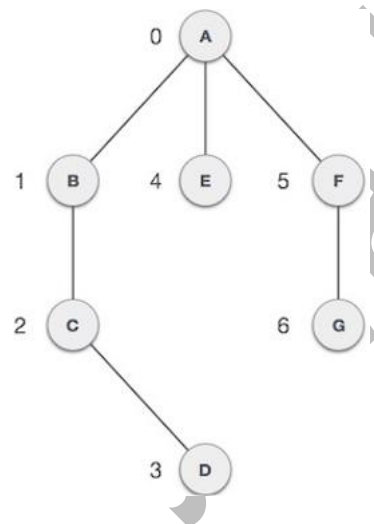
02*Graph Data Structure:

Vertex - Each node of the graph is represented as a vertex. In the following example, the labeled circle represents vertices

Edge - Edge represents a path between two vertices or a line between two vertices. In the following example, the lines from A to B, B to C, and so on represents edges

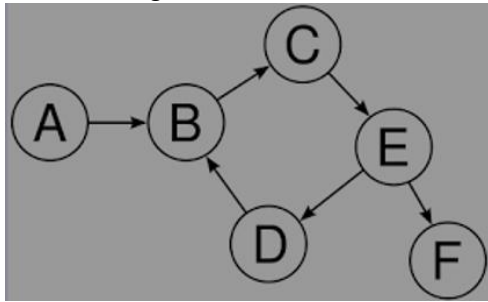
Adjacency - Two nodes or vertices are adjacent if they are connected to each other through an edge. In the following example, B is adjacent to A, C is adjacent to B, and so on.

Path - Represents a sequence of edges between the two vertices. In the following example, ABCD represents a path from A to D



03*Directed Graphs:

Each edge of a directed graph or digraph has a specific direction denoted by arrows. A directed edge is also called unidirectional a directed graph



Directed Graphs Examples:

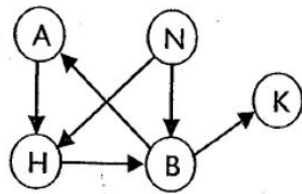


Figure 5.2: A directed graph with 5 nodes and 6 edges

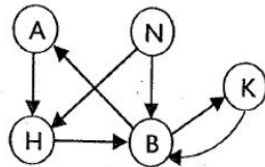
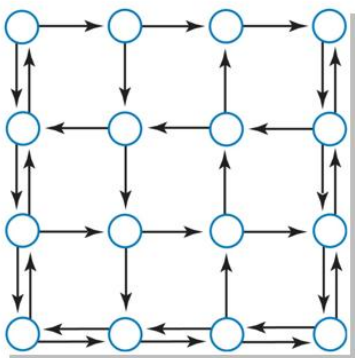
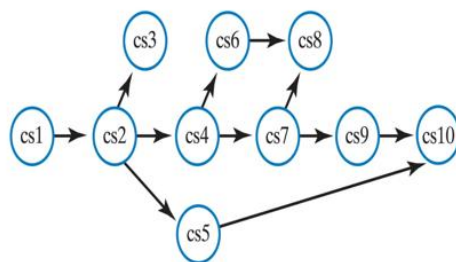


Figure 5.3: Graph with 2 edges connecting the same nodes

Road Maps



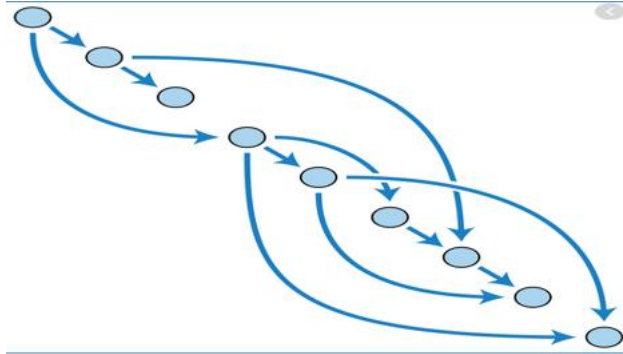
Course Prerequisites



04*Directed Acyclic Graph (DAG):

A path that starts and ends at the same vertex is called a cycle

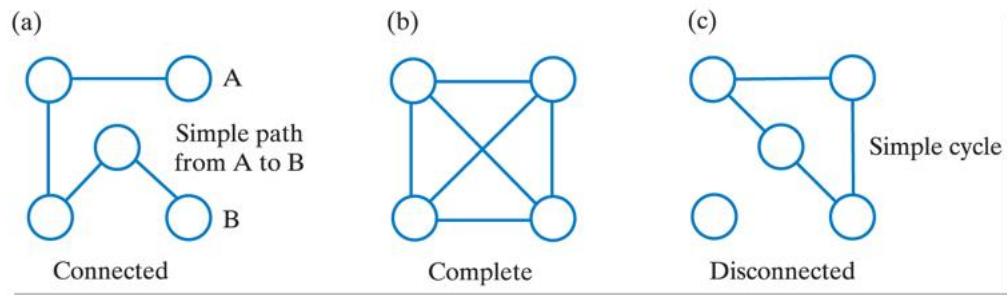
A directed acyclic graph is a directed graph with no cycles



05*Undirected Graphs:

Edges in an undirected graph have no directions

An undirected edge is also called bidirectional



06*Weighted Graphs:

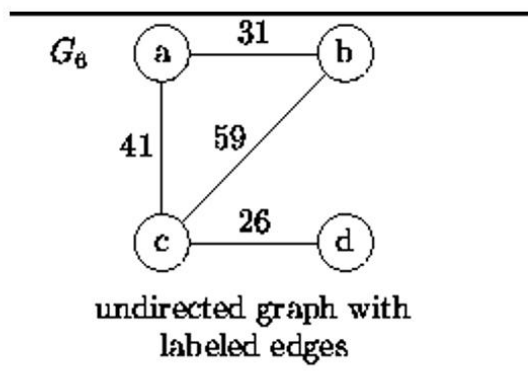
A weighted graph has values (weight / costs) associated with its edges

A path in a weighted graph also has weight which is the sum of the sum of the edge weights

Examples of weights:

- Miles between nodes on a map
- Driving time between nodes
- Taxi cost between node locations

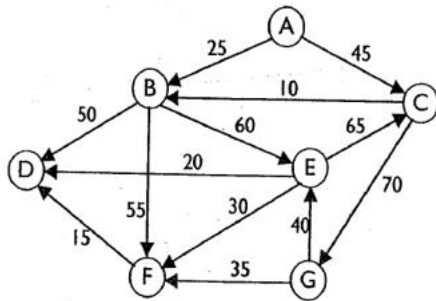
Weighted Graph Example:



07*Graph Representation:

There are two ways to represent a graph:

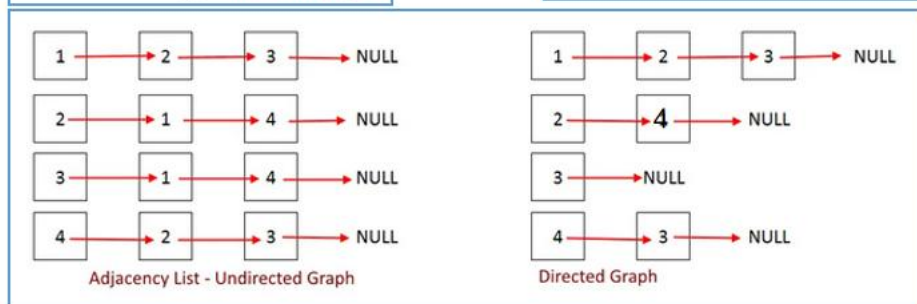
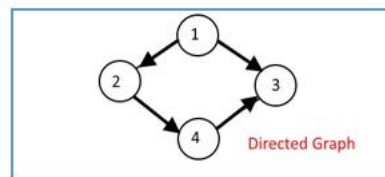
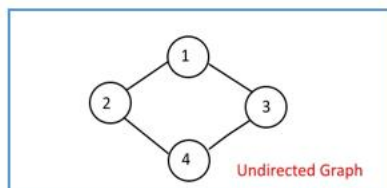
1.Adjacent Matric:



	1	2	3	4	5	6	7
1	0	25	45	∞	∞	∞	∞
2	∞	0	∞	50	60	55	∞
3	∞	10	0	∞	∞	∞	70
4	∞	∞	∞	0	∞	∞	∞
5	∞	∞	65	20	0	30	∞
6	∞	∞	∞	15	∞	0	∞
7	∞	∞	∞	∞	40	35	0

Adjacency matrix, G

2.Adjacency List:



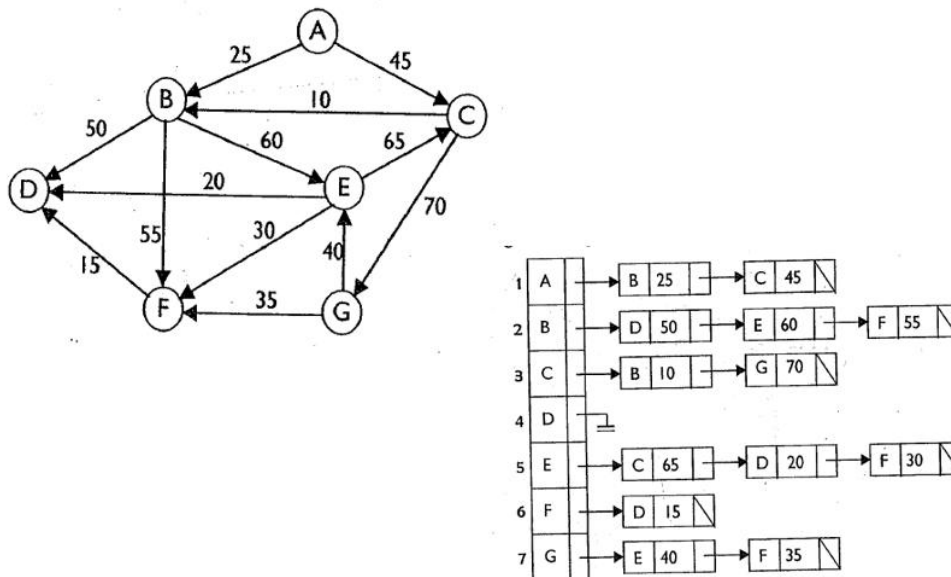
3.Adjacency matrix:

The adjacency matrix is a 2 dimensional array of size $V \times V$, where V is the number of vertices in the graph. It represents the adjacent vertices

4.Adjacency List:

The adjacency List is a one dimensional array of linked lists, where the array size is the same as number of vertices in the graph. Every Vertex in the graph has a LinkedList. Each Node in the LinkedList represents the reference to the other vertices which share an edge with the current vertex. The weights can also be stored in the LinkedList Node

Example of adjacency list:



08*Depth-First Traversal:

Visits a vertex, then A neighbor of the vertex, then A neighbor of the neighbor, etc, advance as much as possible from the original vertex then back up by one vertex. Consider the next neighbor.

The DFS algorithm works as follow :

- 1.Start by putting any one of the graph's vertices on top of a stack
- 2.Take the top item of the stack and add it to the visited list
- 3.Creates a list of that vertex's adjacent nodes. Add the ones which aren't in the visited list to the top of the stack
- 4.Keep repeating steps 2 and 3 until the stack is empty

Depth-First Traversal Example with the stack:

topVertex	nextNeighbor	Visited vertex	vertexStack (top to bottom)	traversalOrder (front to back)
		A	A	A
A	B	B	BA	AB
B	E	E	EBA	ABE
E	F	F	FEBA	ABEF
F	C	C	CFEBA	ABEFC
C			FEBA	
F	H	H	HFEBA	ABEFCH
H	I	I	IHFEBA	ABEFCHI
I			HFEBA	
H			FEBA	
F			EBA	
E			BA	
B			A	
A			A	
	D	D	DA	ABEFCHID
D			DA	
	G	G	GDA	ABEFCHIDG
G			DA	
D			A	
A			empty	ABEFCHIDG

We now look at the other common way of traversing a graph - Breadth-first Traversal.

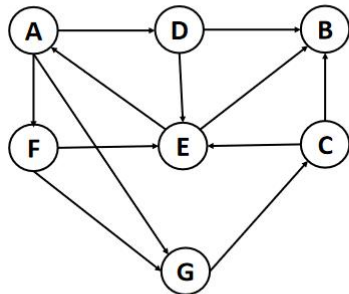
A breadth-first traversal

visits a vertex and then each of the vertex's neighbors before advancing

This is similar to level-order traversal in a binary tree

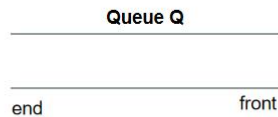
09*Breadth-first Traversal:

We now look at the other common way of traversing a graph - Breath-first Traversal

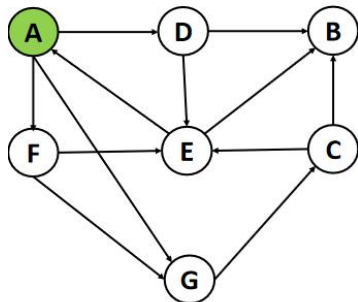


Steps:

Let us look at the details of how a breadth-first search works.

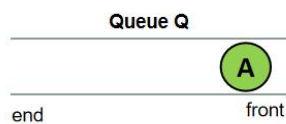


Output:

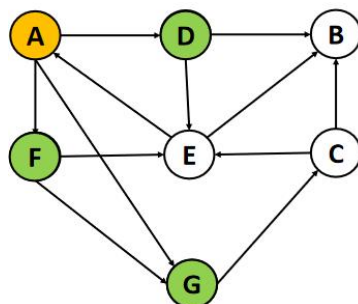


Steps:

Starting from A. Mark and enqueue A.

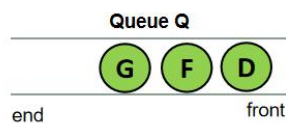


Output:

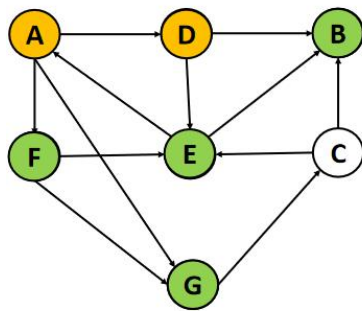


Steps:

Dequeue and Visit A. A is connected to D, F and G via (A,D), (A,F) and (A,G) edges respectively. So mark and enqueue D, F and G.

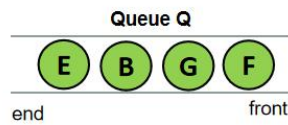


Output: A

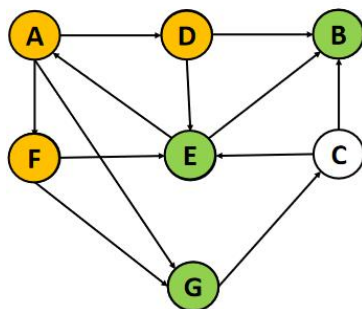


Steps:

Dequeue and Visit D. D is connected to B and E via (D,B), and (D,E) edges respectively.
So mark and enqueue B and E.

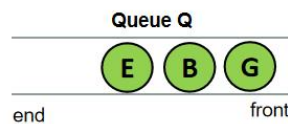


Output: A D

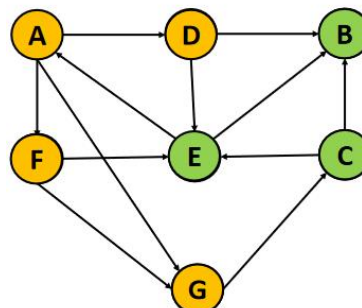


Steps:

Dequeue and Visit F. F is connected to E and G via (F,E) and (F,G) edges respectively. There is no thing to do since E and G have already been seen and equeued to Q.

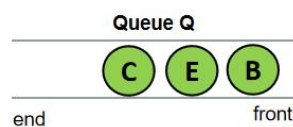


Output: A D F

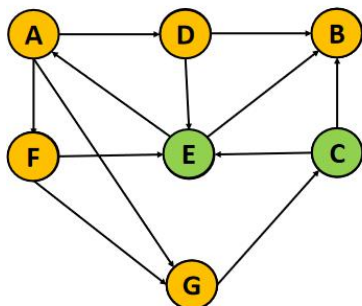


Steps:

Dequeue and Visit G. G is connected to C via (G,C) edge.
Mark and equeue C.

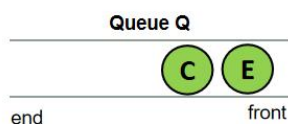


Output: A D F G

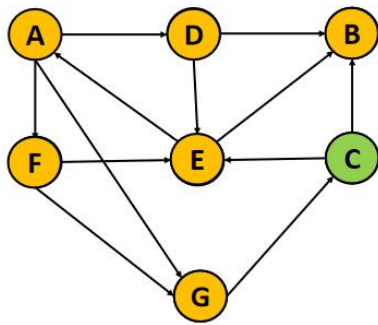


Steps:

Dequeue and Visit B. There are no edges leaving B, so there is nothing to do.

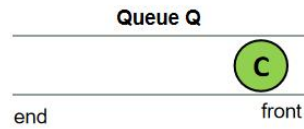


Output: A D F G B

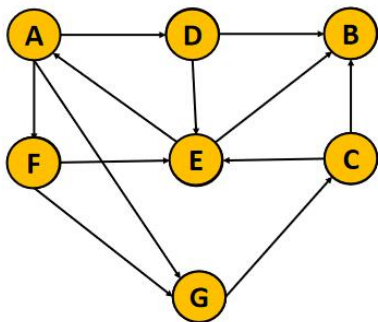


Steps:

Dequeue and Visit E. E is connected to A and B via (E, A) and (E, B) edges respectively. There is no thing to do since A and B have already been visited.

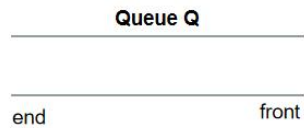


Output: A D F G B E

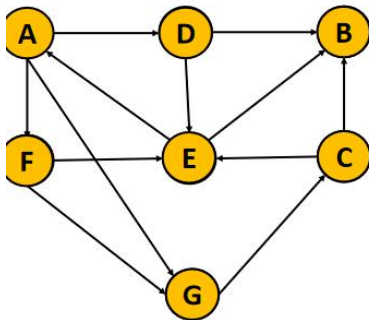


Steps:

Dequeue and Visit C. C is connected to B and E via (C, B) and (C, E) edges respectively. Also there is no thing to do since B and E have already been visited.

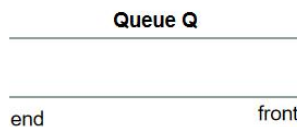


Output: A D F G B E C



Steps:

Q is empty so the algorithm terminates.



Output: A D F G B E C

12* Dijkstra Algorithm:

Dijkstra's algorithm is an algorithm for finding the shortest paths from a source node S to all other nodes in a graph

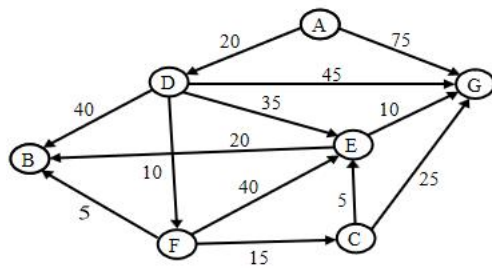
Shortest Path:

Suppose we want to find the shortest path from A to E

A → D → E (with a cost of 55)

A → D → F → E (cost 70)

A → D → F → C → E (cost 50)



Important observation : while we may be interested in finding the shortest path from one vertex to another specific vertex , it turns out that it has the same complexity of finding the shortest path from the source vertices

How Dijkstra works:

We first assume that the weights of edges are non-negative

We want to find the shortest path from s to all other vertices in the graph

We grow a “cloud” of vertices, beginning with s and eventually covering all the vertices

The distance of a vertex v a label $d(v)$ is the length of a shortest path between s and v

We store with each vertex v a label $d(v)$ representing the distance of v from s in the subgraph consisting of the cloud and its adjacent vertices

At each step

We add to the cloud the vertex u outside the cloud with the smallest distance label, $d(u)$

We update the labels of the vertices adjacent to u

13* Algorithm:

1) Create a set Marked that keeps track of already visited vertices. Initially , this set is empty

2) Assign a distance value to all vertices in the graph . Initialize all distance value as INFINITE. Assign distance value as 0 for the source so that it is picked first

3) While Marked doesn't include all vertices

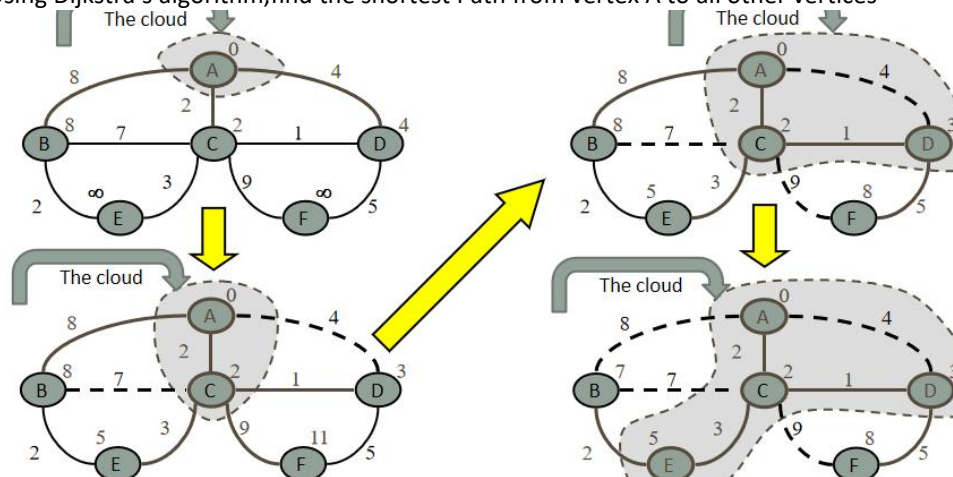
a- pick a vertex u which is not there in Marked and has a minimum distance value

b- Include u to Marked

c- Update distance value of all adjacent vertices of u. To update the distance values . iterate through all adjacent vertices. For every adjacent vertex v , if the sum of distance value of u (from source) and weight of edge u-v , is less than the distance value of v , then update the distance value of v

Example:

Using Dijkstra's algorithm, find the shortest Path from vertex A to all other vertices



Shortest Path from A to all other vertices:

A to B : A -> C -> E -> B with cost = 7

A to C: A -> C with cost = 2

A to D: A -> C -> D with cost = 3

A to E: A -> C -> E with cost = 5

A to F: A -> C -> D -> F with cost = 8

Chapter 07: HashTable & HashMap

01*What is a Hash Table?

A Hash Table is a data structure used to store data in the form of key-value pairs

Each key is associated with exactly one value

The main purpose of a hash table is to provide very fast access to data

Example idea(conceptual):

Key: "Jawad" -> Value: 1200

Key: "Ali" -> Value: 1000

02*Why Hash Tables Are Fast?

-Hash tables do not search data sequentially like arrays of lists

-Instead, they use a hash function to calculate a position(index) where the data should be stored

-In most cases, the time complexity is $O(1)$

-Because of this: Insertion is fast, Searching is fast, Deletion is fast

03*Hash Function:

-A hash function converts a key into an integer value called a hash code

-The hash code is then converted into an array index

General idea:

$\text{index} = \text{hashCode}(\text{key}) \% \text{arraySize}$

04*Collision:

-A collision happens when two different keys produce the same index

-This is normal and expected in hash tables

-Java handles collisions using chaining:

Each index in the internal array can store multiple key-value pairs

These pairs are stored in a linked structure

05*HashMap vs HashTable:

HashTable is the old version : Thread-safe / slower / Does not allow null keys or null values

HashMap is the modern version : Not thread-safe by default / Faster / Allows one null key and multiple null values

In modern Java , HashMap is preferred

06*HashMap:

HashMap is a class from the java.util package

Declaration:

```
HashMap<KeyType, ValueType> map = new HashMap<>();
```

Example:

```
HashMap<Integer , String> students = new HashMap<>();
```

07*Inserting Data into HashMap:

Using put() :

Each key must be unique

If you insert a value with an existing key , the old value is replaced

```
students.put(1 , "Jawad");
```

```
students.put(2 , "Ali");
```

```
students.put(3 , "Hassan");
```

08*Retrieving Data from HashMap:

Using get():

```
System.out.println(students.get(2));
```

Output

Ali

If the key doesn't exist , get() return null

09*Checking Keys and Values:

To check if a key exists:

```
students.containsKey(1);
```

To check if a value exists:

```
students.containsValue("Jawad");
```

Both return either true or false;

10*Removing Data:

```
Student.remove(2);
```

```
System.out.println(students);
```

Output:

```
{1=Jawad, 3=Hassan}
```

11*Iterating Over HashMap:

One way is by using the keys:

```
for(Integer key: student.keySet()) {  
    System.out.println(key + "->" + students.get(key));  
}
```

12*Null in HashMap:

HashMap allows:

One null key

Multiple null values

Example:

```
HashMap<String, Integer> map = new HashMap<>();  
map.put(null, 5);  
map.put("A", null);  
System.out.println(map);
```

Output

{null = 5, A=null}

13*Internal Structure:

-Internally, HashMap uses:

An array of buckets

Each bucket stores entries

Uses hashing to decide the bucket

14*When to Use HashMap:

-Use HashMap when:

Data is stored as key-value pairs

Fast access is required

Order of elements does not matter

-Do not use HashMap if:

You need sorted keys

You need insertion order

Chapter 08 : Vector

01*Introduction to Vector:

Vector is one of the data structures provided by Java. It is used to store multiple elements in a single object.

A Vector works like an array, but it has a dynamic size. This means that the Vector can automatically grow when more elements are added.

Vector is part of the Java Collections Framework and is found in the java.util package.

Example:

```
Vector<Integer> numbers = new Vector<>();  
numbers.add(10);  
numbers.add(20);  
numbers.add(30);  
System.out.println(numbers);
```

Output:

[10, 20, 30]

02*Definition of Vector:

A Vector is a dynamic array that stores elements in a continuous memory structure and automatically increases its capacity when the current storage is full.

Vector stores elements in an ordered way.

Each element has an index.

Index starts from 0.

Vector size can change during program execution.

Vector can store objects of a specific data type using Generics.

Accessing:

```
v.get(1);
```

Output:

```
10
```

03*Features of Vector:

1.Dynamic Size:

Unlike arrays, Vector size is not fixed.

Array:

```
int arr[] = new int[3];
```

The size is always: 3

Vector:

```
Vector<Integer> v = new Vector<>();
```

The size can become:

3 → 10 → 20 → ... depending on the number of elements.

2.Stores Multiple Elements:

A Vector can store many values of the same type.

Example:

```
Vector<String> names = new Vector<>();  
names.add("Ali");  
names.add("Ahmad");  
names.add("Sara");
```

Output:

```
[Ali, Ahmad, Sara]
```

3.Index-Based Access:

Each element has an index.

Example:

Index:	0	1	2
	Ali	Ahmad	Sara

Access:

```
names.get(2);
```

Output:

```
Sara
```

4.Thread Safety:

Vector methods are synchronized, which means they are designed to support multiple threads accessing the Vector safely.

However, this synchronization can make Vector slower compared with other dynamic arrays like ArrayList.

04*Why Use Vector?

We use Vector when we need a collection that can change its size during execution.

Problem with Normal Arrays

Arrays have a fixed size.

Example:

```
int numbers[] = new int[3];
```

Memory:

10 20 30

If we want to add another value:

40

there is no available space.

We must create a new larger array and copy all elements.

Solution:

Vector handles resizing automatically.

Before adding:

Capacity = 3

10 20 30

After adding a new element:

Capacity increases

10 20 30 40

05*Vector Structure:

A Vector internally uses an array to store its elements.

The structure of a Vector contains three important parts:

1.Internal Array:

Vector uses an internal array as the actual storage.

Example:

```
Vector<Integer> v = new Vector<>();  
v.add(10);  
v.add(20);  
v.add(30);
```

The empty spaces are available for future elements.

2.Size:

Size represents the number of elements currently stored in the Vector.

Example:

```
Vector<Integer> v = new Vector<>();  
v.add(5);  
v.add(10);  
v.add(15);
```

Size:

size = 3

```
System.out.println(v.size());
```

Output:

3

3.Capacity:

Capacity represents the total number of elements that the Vector can store before it needs to increase its size.

Example:

Capacity = 5

Size = 3

Here:

size = 3

capacity = 5

The Vector has two empty spaces available.

```
System.out.println(v.capacity());
```

4.Dynamic Resizing:

When the Vector becomes full, it automatically creates a larger array and copies the old elements.

Example:

Before:

Capacity = 3

1	2	3
---	---	---

Adding:

40

After resizing:

Capacity = 6

06*Creating a Vector:

Before using Vector, we need to import the Vector class.

```
import java.util.Vector;
```

1.Creating an Empty Vector:

Syntax:

```
Vector<DataType> name = new Vector<>();
```

Example:

```
Vector<Integer> numbers = new Vector<>();
```

Output:

[]

The Vector is created but contains no elements.

2.Creating Vector with Initial Capacity:

We can specify the initial capacity.

Example:

```
Vector<Integer> numbers = new Vector<>(10);
```

Capacity = 10

Size = 0

The Vector has space for 10 elements initially.

3.Creating Vector with Initial Values:

We can add values after creating the Vector.

Example:

```
Vector<String> names = new Vector<>();  
names.add("Ali");  
names.add("Sara");  
names.add("Omar");  
System.out.println(names);
```

Output:

```
[Ali, Sara, Omar]
```

07*Adding Elements to Vector:

Vector provides methods to insert new elements.

add() :

The add() method adds an element at the end of the Vector.

Syntax:

```
vectorName.add(element);
```

Example:

```
Vector<Integer> numbers = new Vector<>();  
numbers.add(10);  
numbers.add(20);  
numbers.add(30);  
System.out.println(numbers);
```

Output:

```
[10, 20, 30]
```

The order of elements is maintained.

add(index, element):

The add() method can also insert an element at a specific index.

Syntax:

```
vectorName.add(index, element);
```

Example:

```
Vector<String> names = new Vector<>();  
names.add("Ali");  
names.add("Ahmad");  
names.add("Sara");  
names.add(1,"Omar");  
System.out.println(names);
```

Output:

```
[Ali, Omar, Ahmad, Sara]
```

08*Accessing Vector Elements:

To access elements inside a Vector, we use the get() method.

Syntax:

```
vectorName.get(index);
```

Example:

```
Vector<Integer> numbers = new Vector<>();  
numbers.add(100);  
numbers.add(200);  
numbers.add(300);  
System.out.println(numbers.get(1));
```

Output:

```
200
```

09*Updating Vector Elements:

To change an existing element inside a Vector, we use the set() method.

The set() method replaces the value at a specific index.

Syntax:

```
vectorName.set(index, newValue);
```

Example:

```
import java.util.Vector;  
public class Main {  
    public static void main(String[] args) {  
        Vector<Integer> numbers = new Vector<>();  
        numbers.add(10);  
        numbers.add(20);  
        numbers.add(30);  
        System.out.println(numbers);  
        numbers.set(1, 50);  
        System.out.println(numbers);  
    }  
}
```

Output:

```
[10, 20, 30]
```

```
[10, 50, 30]
```

10*Removing Elements from Vector:

remove(index):

Removes an element using its index.

Syntax:

```
vectorName.remove(index);
```

Example:

```
Vector<Integer> numbers = new Vector<>();  
numbers.add(10);  
numbers.add(20);  
numbers.add(30);  
numbers.remove(1);  
System.out.println(numbers);
```

Output:

```
[10, 30]
```

remove(Object):

Removes a specific value from the Vector.

Example:

```
Vector<String> names = new Vector<>();
names.add("Ali");
names.add("Ahmad");
names.add("Sara");
names.remove("Ahmad");
System.out.println(names);
```

Output:

```
[Ali, Sara]
```

clear():

Removes all elements from the Vector.

Example:

```
Vector<Integer> numbers = new Vector<>();
numbers.add(10);
numbers.add(20);
numbers.clear();
System.out.println(numbers);
```

Output:

```
[]
```

11*Searching in Vector:

Vector provides methods to search for elements.

contains():

checks if an element exists in the Vector. (return true or false)

Syntax:

```
vectorName.contains(element);
```

Example:

```
Vector<String> names = new Vector<>();
names.add("Ali");
names.add("Sara");
System.out.println(names.contains("Ali"));
System.out.println(names.contains("Omar"));
```

Output:

```
true
false
```


indexOf() :

Returns the index of an element.

Syntax:

```
vectorName.indexOf(element);
```

Example:

```
Vector<Integer> numbers = new Vector<>();  
numbers.add(5);  
numbers.add(10);  
numbers.add(15);  
System.out.println(numbers.indexOf(10));
```

Output:

```
1
```

12* Vector Size and Capacity Methods:

Vector provides methods to get information about its storage.

size():

Returns the number of elements currently stored.

Example:

```
Vector<Integer> numbers = new Vector<>();  
numbers.add(10);  
numbers.add(20);  
numbers.add(30);  
System.out.println(numbers.size());
```

Output:

```
3
```

capacity():

Returns the total available storage capacity.

Example:

```
Vector<Integer> numbers = new Vector<>(10);  
System.out.println(numbers.capacity());
```

Output:

```
10
```

isEmpty():

Checks whether the Vector contains elements or not. (return true or false)

Example:

```
Vector<Integer> numbers = new Vector<>();  
System.out.println(numbers.isEmpty());  
numbers.add(5);  
System.out.println(numbers.isEmpty());
```

Output:

```
true  
false
```